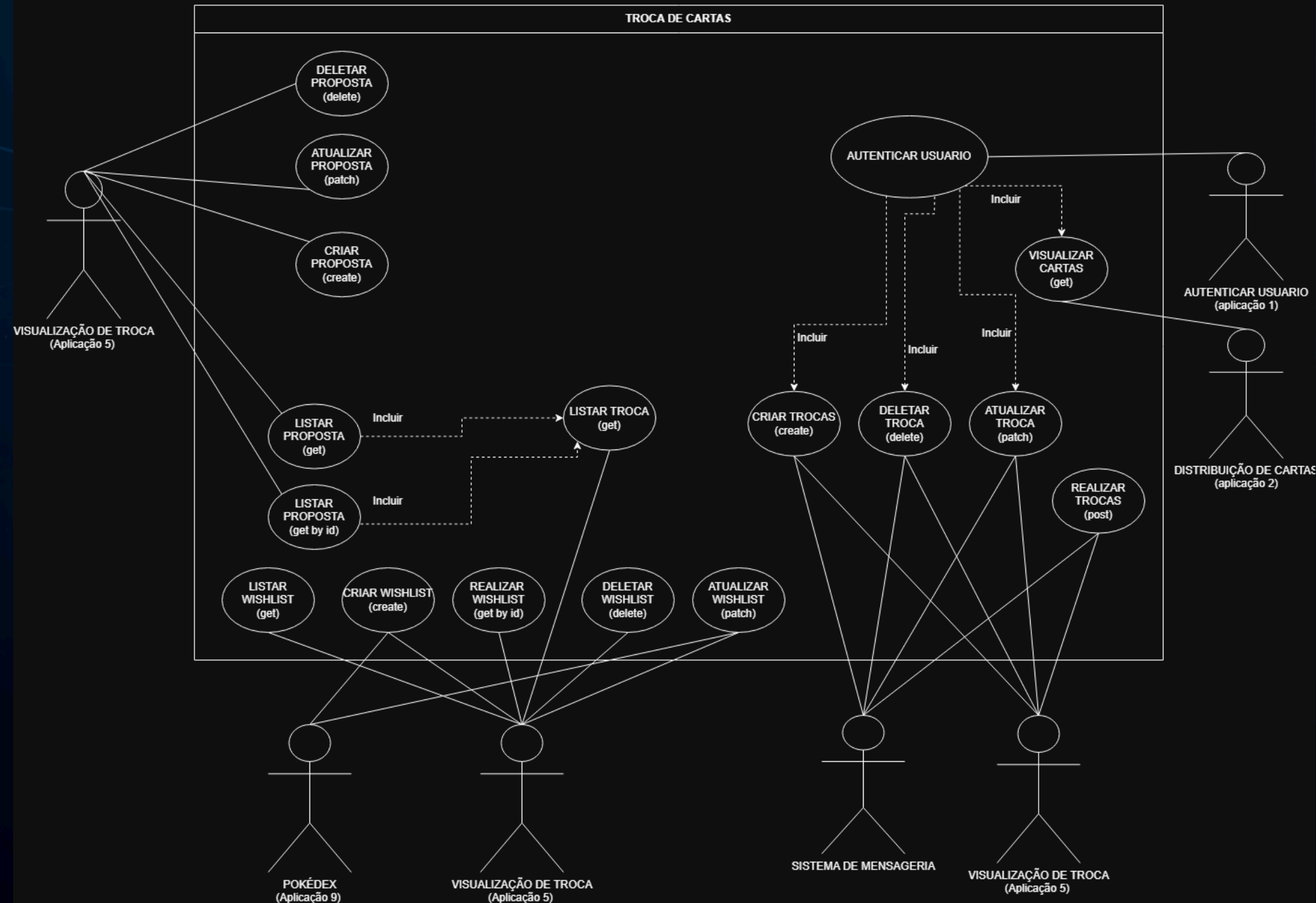
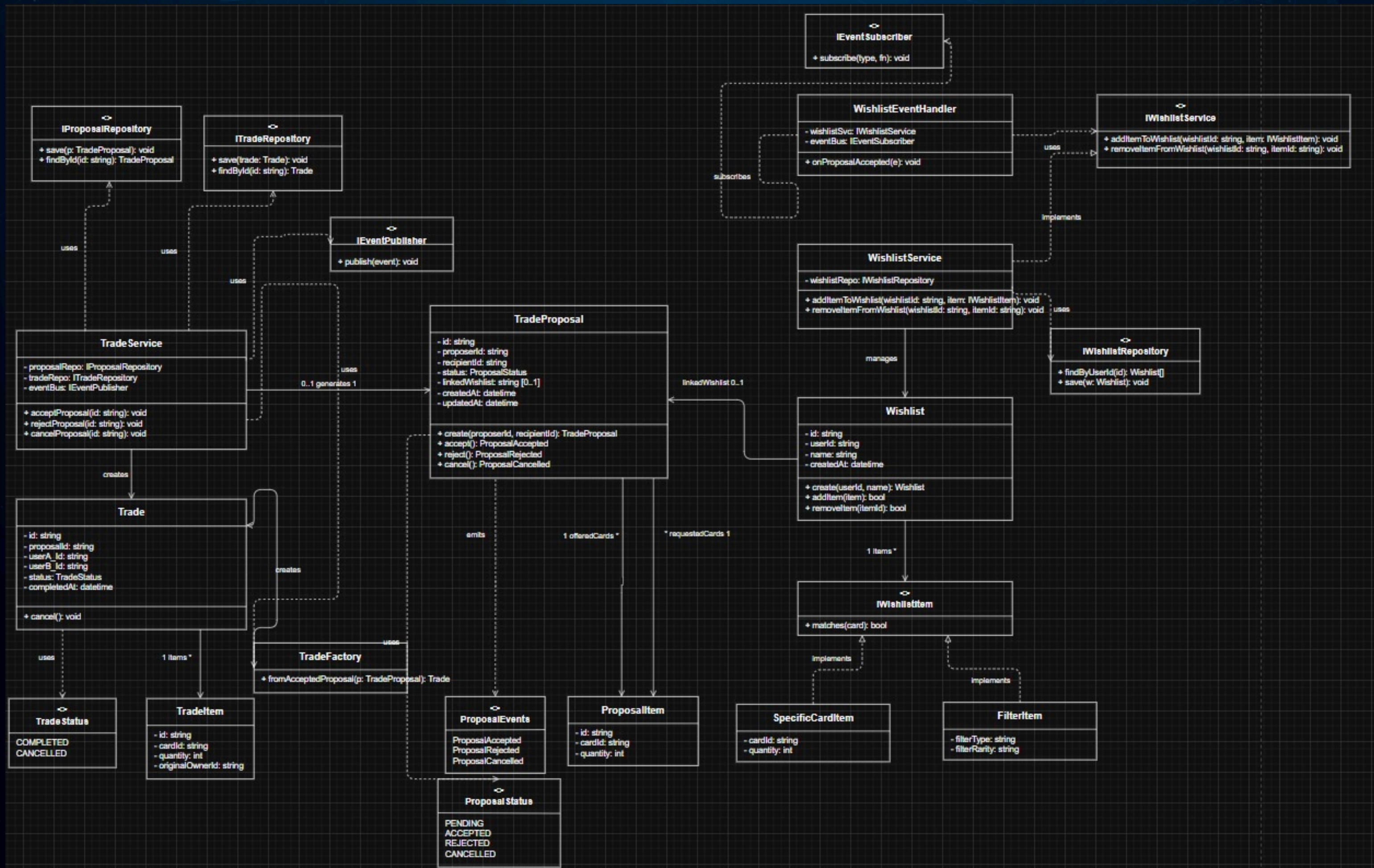




REFACTOR COM SOLID

DIAGRAMA DE USO







PRINCIPAIS PROBLEMAS

01

- **Responsabilidade dupla**

Serviços acumulavam criação de entidade e orquestração de fluxo ao mesmo tempo

02

- **Dependência concreta**

Classes de alto nível conheciam diretamente repositórios e implementações de eventos

03

- **Baixa extensibilidade**

Adicionar um novo tipo de item exigia alterar código central já funcionando



SRP · CONCEITO

Cada classe deve ter um único motivo para mudar.

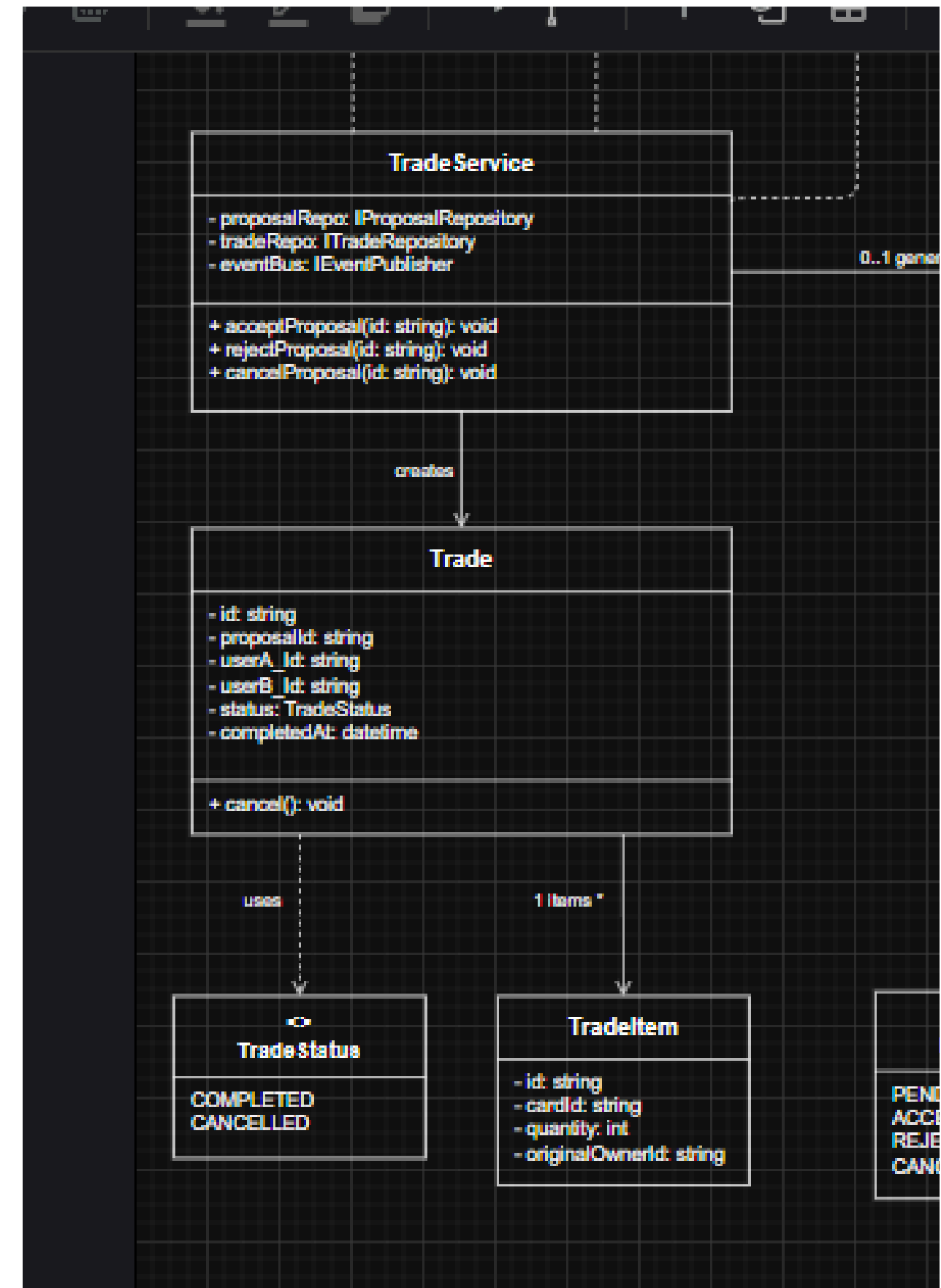
Classes afetadas (lista):

- TradeFactory
- TradeService
- WishlistService
- WishlistEventHandler
- TradeProposal

SRP • NA PRÁTICA

Antes da correção:

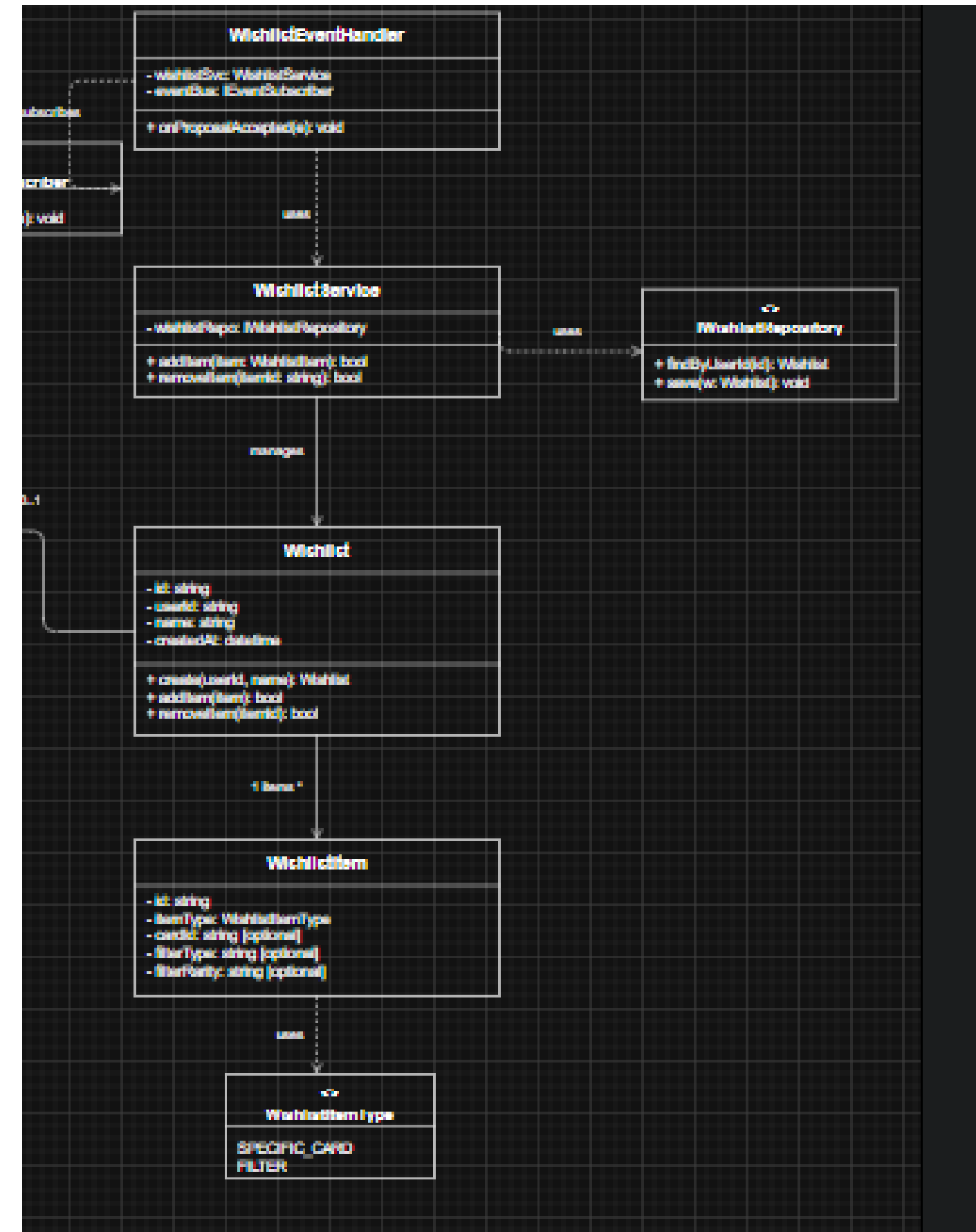
- **TradeService** criava o objeto **Trade** e orquestrava o fluxo
- **WhislitsService** misturava persistência, regra de negócio e eventos
- Difícil testar partes isoladas



SRP · NA PRÁTICA

Antes da correção:

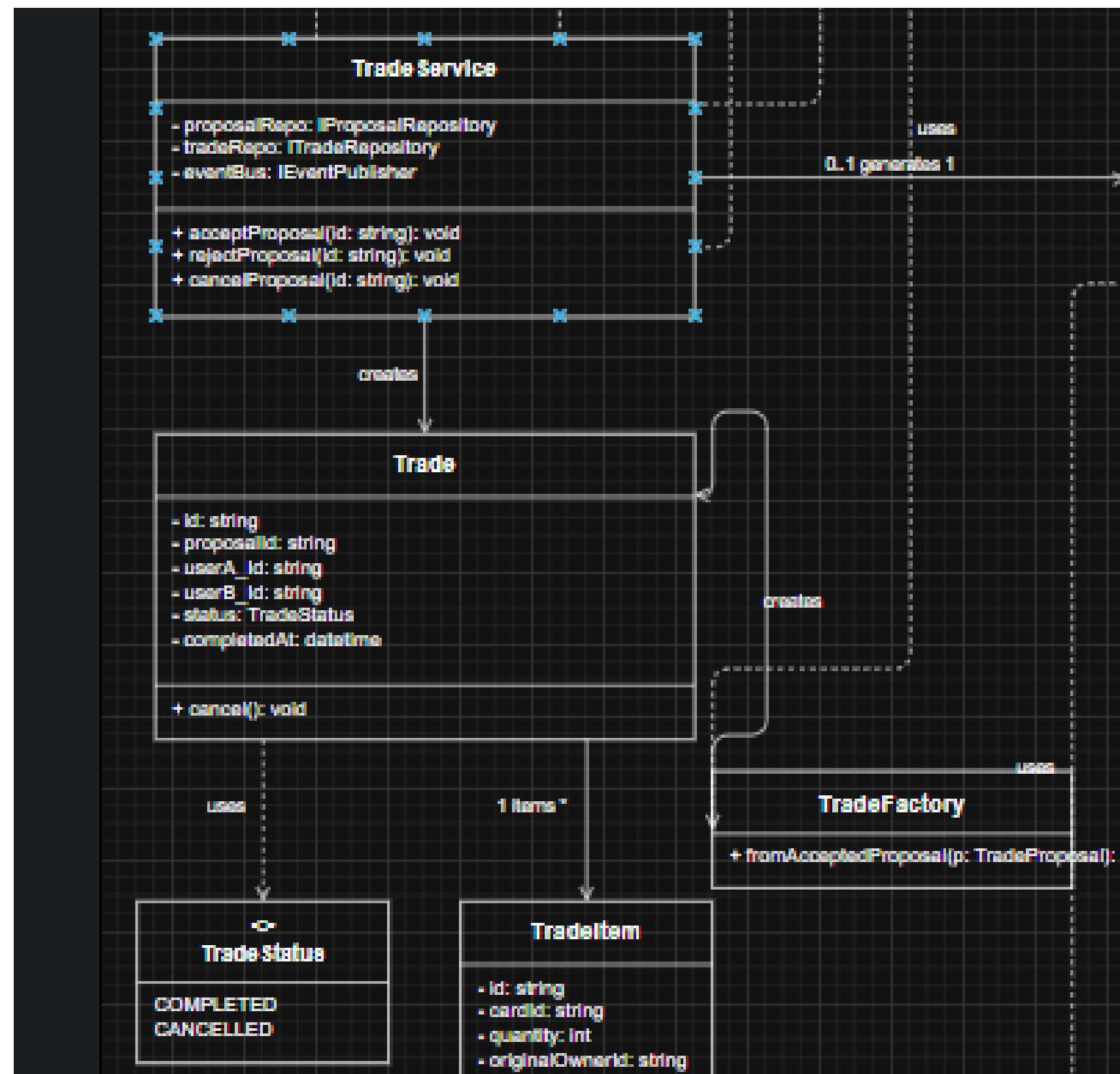
- **TradeService** criava o objeto **Trade** e orquestrava o fluxo
- **WhislitsService** misturava persistência, regra de negócio e eventos
- Difícil testar partes isoladas



SRP • NA PRÁTICA

Depois da correção:

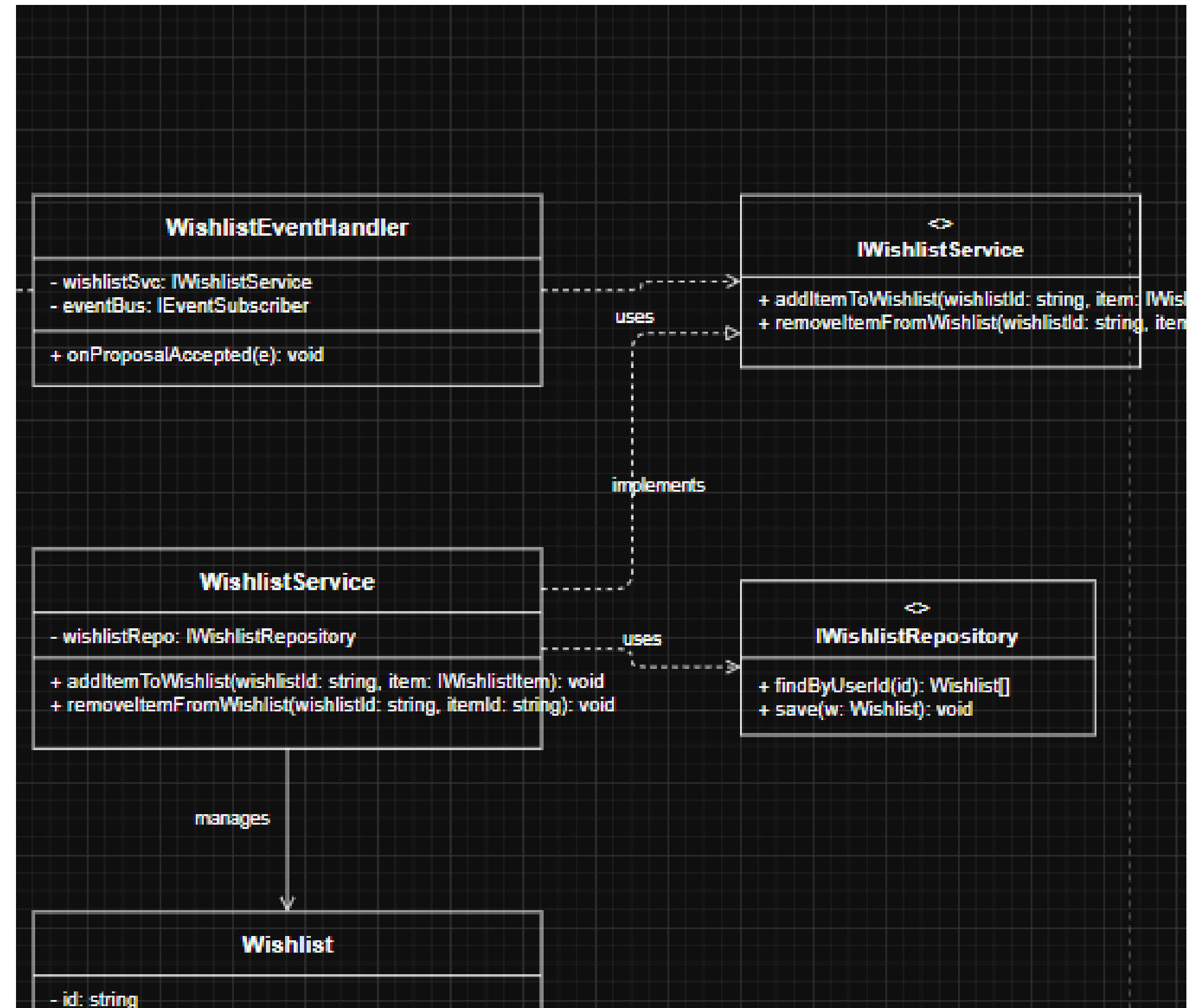
- **TradeFactory** isola a criação do objeto **Trade**
- **TradeService** apenas orchestra aceitar / rejeitar / cancelar
- **WishlistEventHandler** cuida só de reagir a eventos



SRP • NA PRÁTICA

Depois da correção:

- **TradeFactory** isola a criação do objeto **Trade**
- **TradeService** apenas orchestra aceitar / rejeitar / cancelar
- **WishlistEventHandler** cuida só de reagir a eventos



OCP · NA PRÁTICA

OCP — Extensão por composição

Para suportar um novo tipo de item, basta criar outra implementação de `IWishlistItem` — sem alterar `WishlistService` nem `Wishlist`

Class

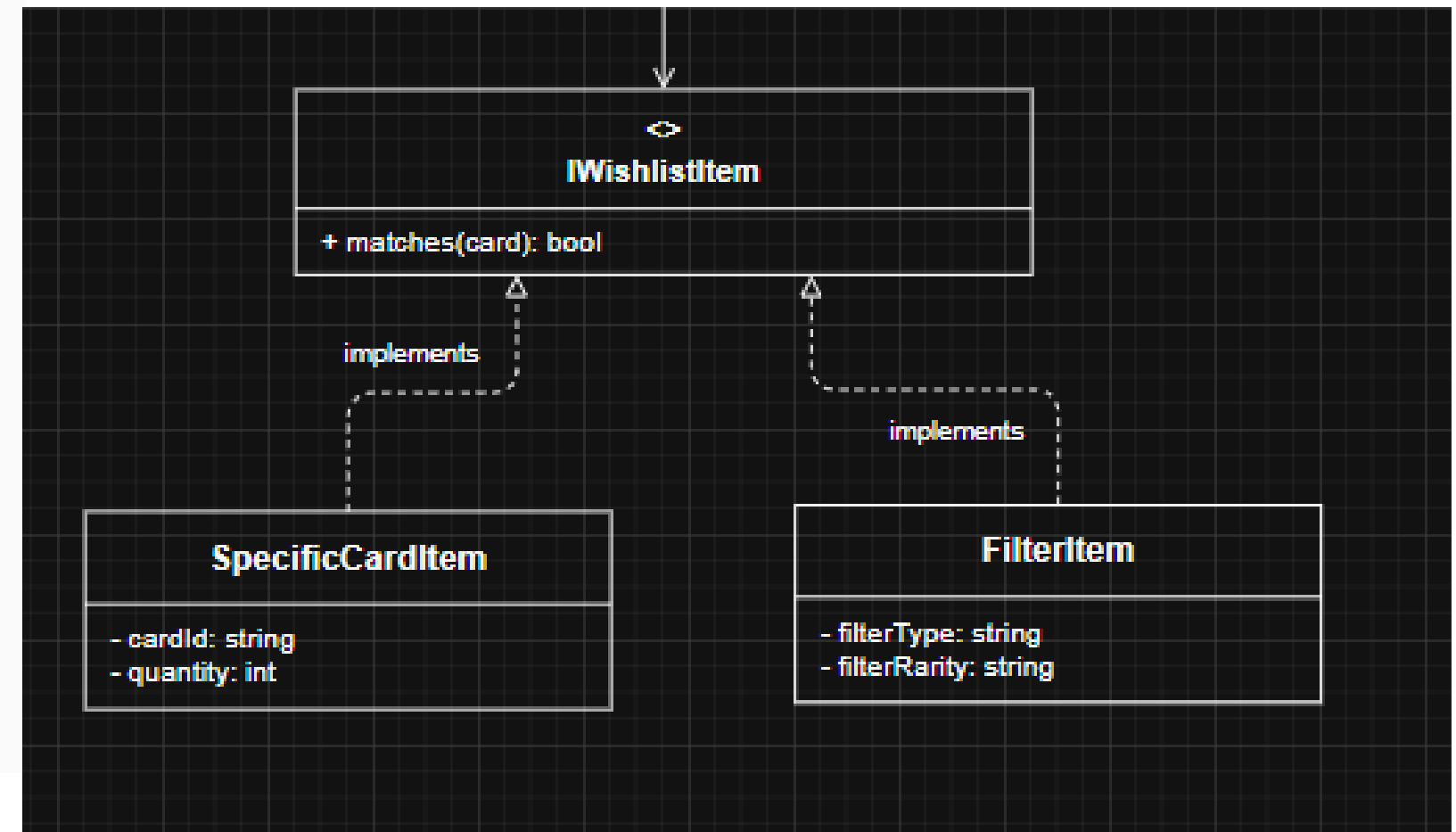
Class

01

Interface `IWishlistItem` → Define método `matches(card): bool` → Aberta para extensão → **Fechada para modificação**

02

Classes Implementadoras → `SpecificCardItem : IWishlistItem` → `FilterItem : IWishlistItem` → Extensão sem alteração

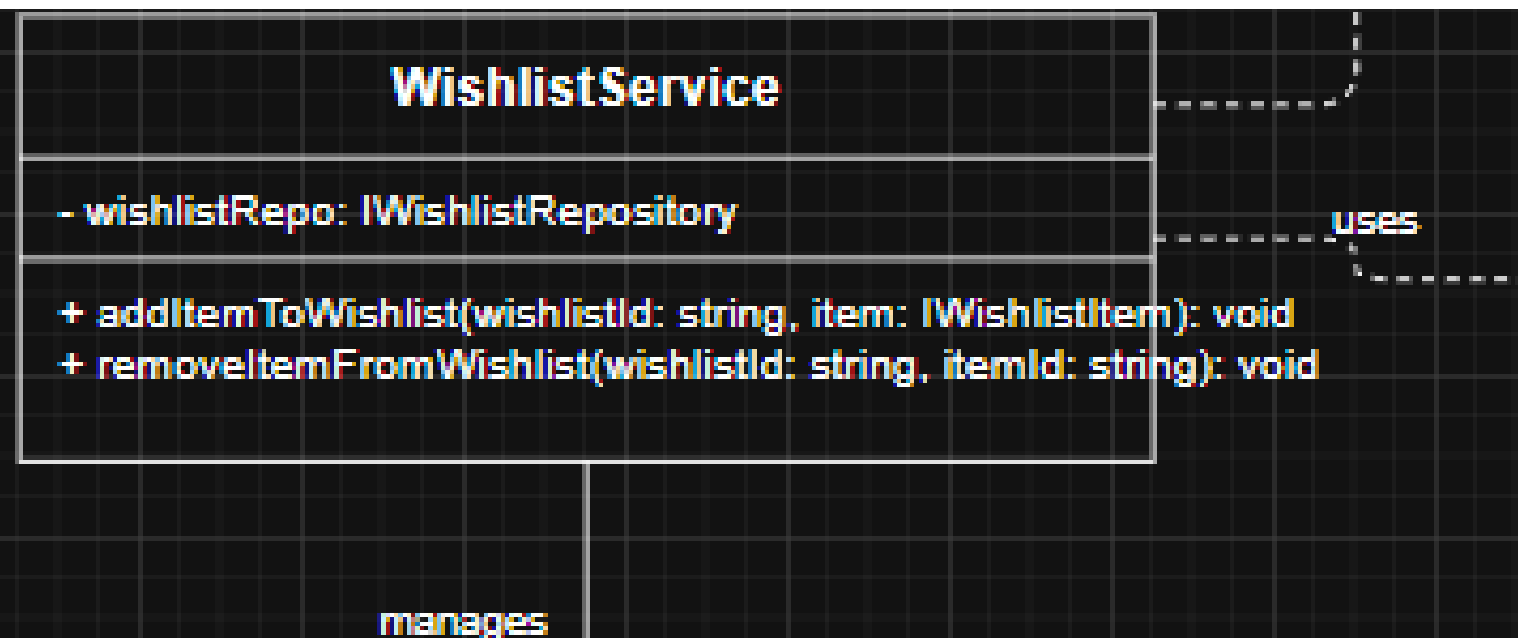


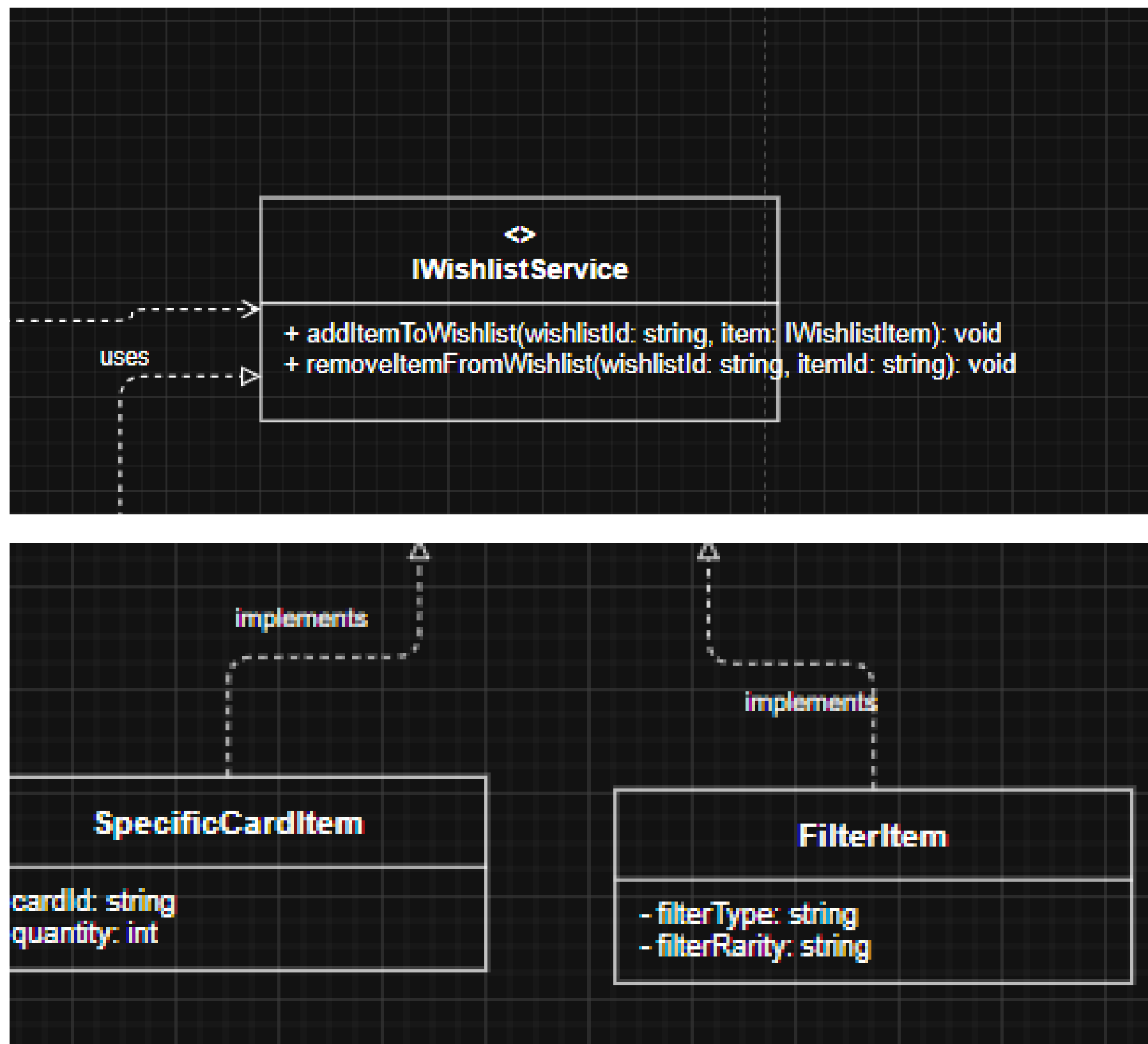
O • OPEN/CLOSED PRINCIPLE

Aberto para extensão, fechado para modificação.

Onde aparece:

- **WishlistService.addItemToWishlist(wishlistId, item: IWishlistItem)**
- **Wishlist** se relaciona com itens via **IWishlistItem**
- Interface **IWishlistItem**
- **SpecificCardItem** e **FilterItem**



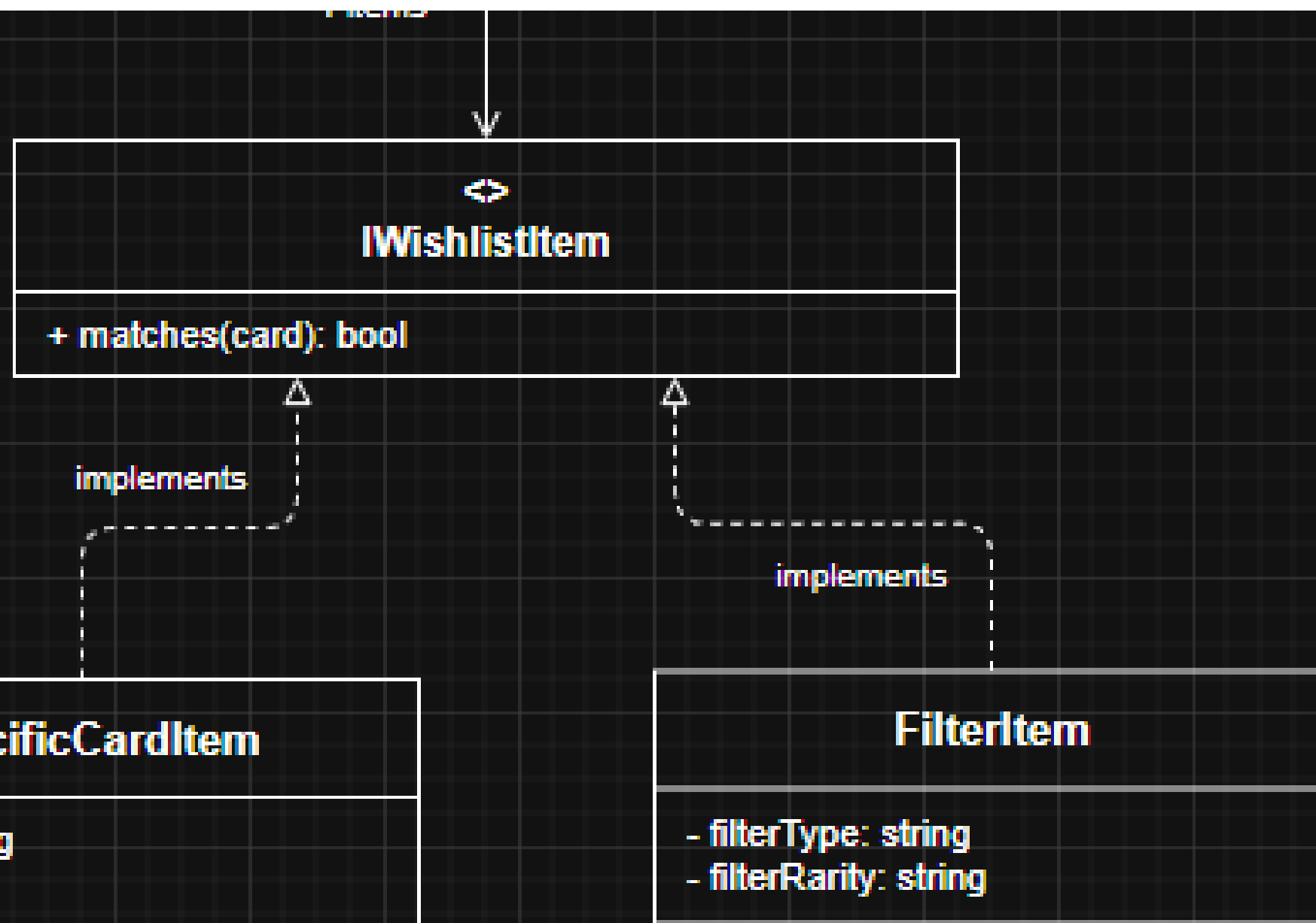


O · OPEN/CLOSED PRINCIPLE

Aberto para extensão, fechado para modificação.

Onde aparece:

- **WishlistService.addItemToWishlist(wishlistId, item: IWishlistItem)**
- **Wishlist** se relaciona com itens via **IWishlistItem**
- Interface **IWishlistItem**
- **SpecificCardItem** e **FilterItem**



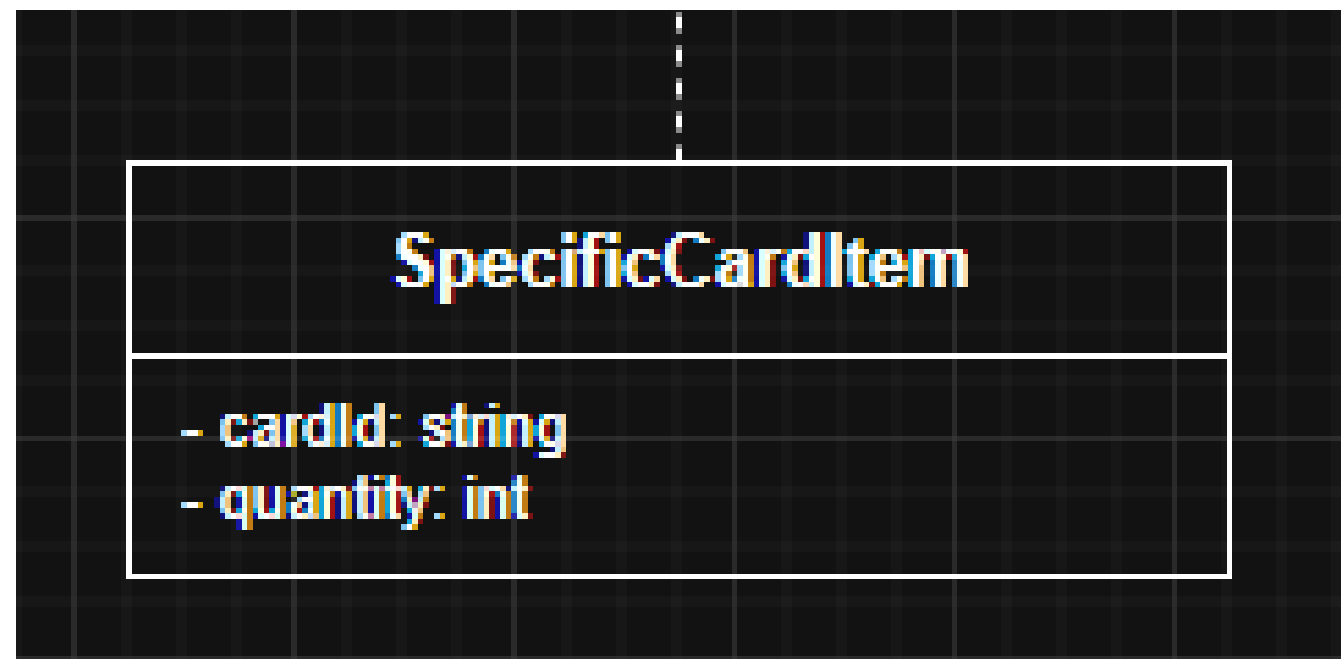
LSP • CONCEITO

Implementações concretas devem poder substituir a abstração sem quebrar o comportamento esperado.

Onde aparece:

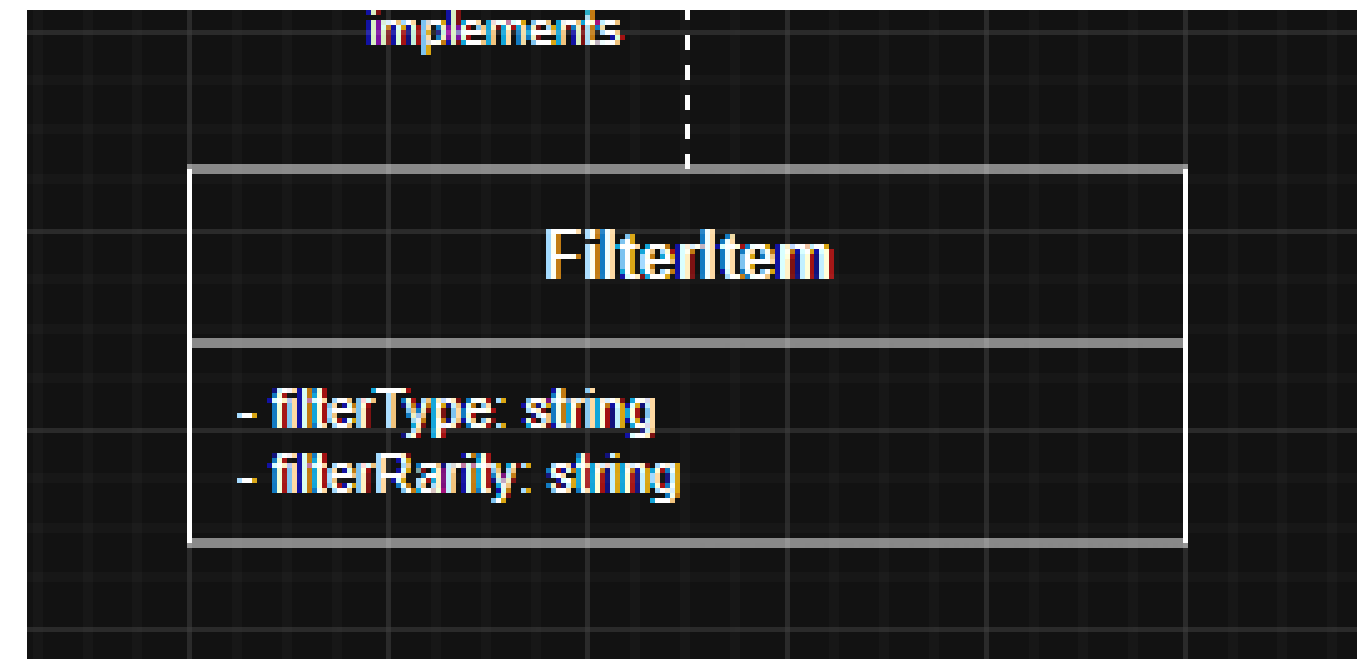
- **SpecificCardItem** implementa **IWishlistItem**
- **FilterItem** implementa **IWishlistItem**
- Contrato definido por: **matches(card): bool**

LSP · NA PRÁTICA



SpecificCardItem · matches(card)

Verifica se o card é exatamente aquele item



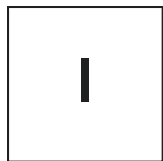
FilterItem · matches(card)

Verifica se o card atende ao filtro definido

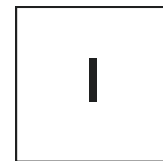
Wishlist e WishlistService tratam os dois de forma uniforme. Trocar um pelo outro no ponto de uso não quebra nada



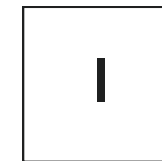
I • INTERFACE SEGREGATION PRINCIPLE



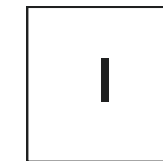
IWishlistService — expõe
só operações de wishlist



IWishlistRepository —
contrato pequeno focado
em persistência



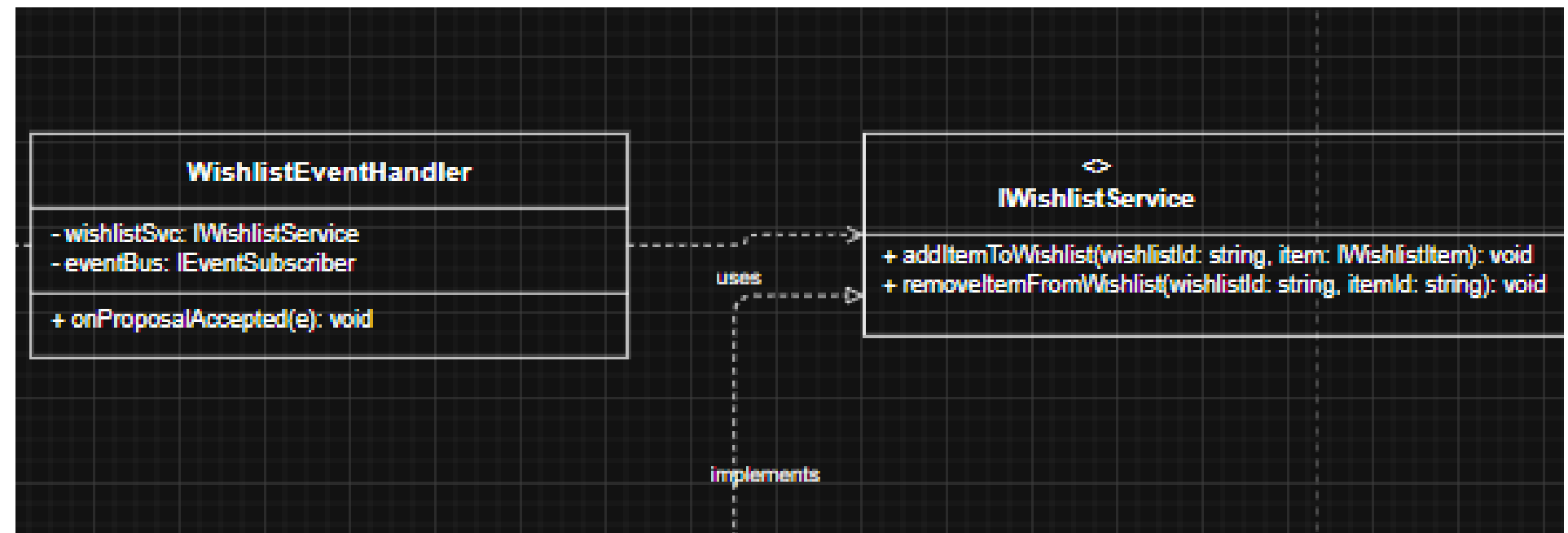
IEventPublisher — só
publica

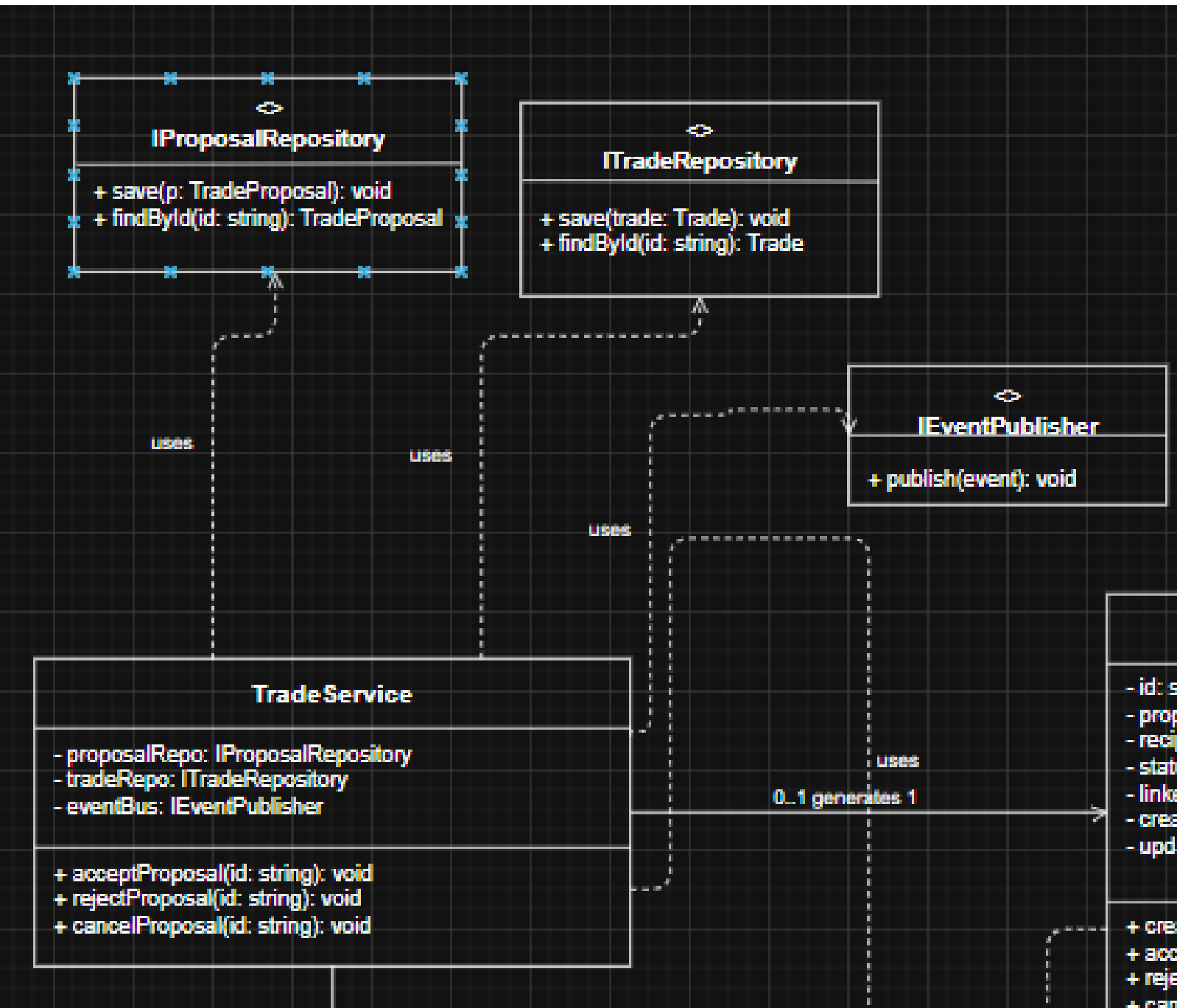


IEventSubscriber — só se
inscreve

ISP • NA PRÁTICA

WishlistEventHandler depende de **IWishlistService**, não da classe concreta inteira. Quem publica não precisa conhecer inscrição, e vice-versa.



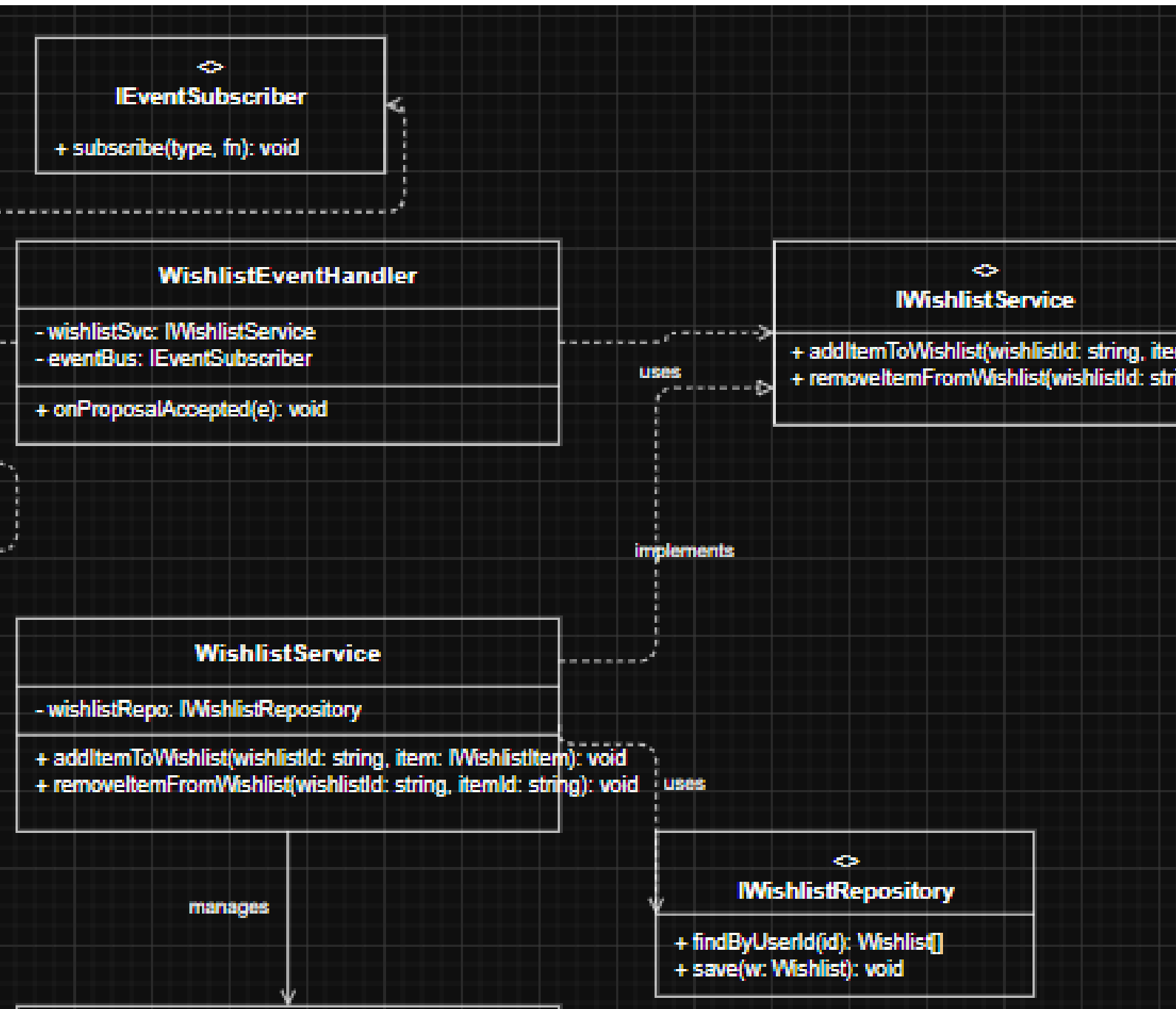


DIP • NA PRÁTICA

Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações

Onde aparece:

- TradeService → IProposalRepository, ITradeRepository, IEventPublisher
- WishlistEventHandler → IWishlistService, IEventSubscriber
- WishlistService → IWishlistRepository



DIP • NA PRÁTICA

Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações

Onde aparece:

- TradeService → IProposalRepository, ITradeRepository, IEventPublisher
- WishlistEventHandler → IWishlistService, IEventSubscriber
- WishlistService → IWishlistRepository



O QUE MUDOU NA NOVA ATUALIZAÇÃO

SRP

Criação de TradeFactory

**DIP
ISP**

Introdução de IWishlistService

**OCP
LSP**

Introdução de IWishlistItem

**ISP
DIP**

Separação IEventPublisher / IEventSubscriber

**OCP
LSP**

SpecificCardItem e FilterItem

DIP

Repositórios por interface